Pandas

- Jake VanderPlas. 2016. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, Inc.
-    3. kapitulua - Datuen manipulazioa Pandas-ekin
- https://github.com/jakevdp/PythonDataScienceHandbook

Pandas-ek honako hauek eskaintzen ditu:

- Sarrera/irteerako gaitasun aberatsak (CSV, Excel, SQL, JSON eta abarretatik datuak irakurri eta horietan idaztea)
- Dimentsio bateko (**Series**) eta bi dimentsioko taula-formako (**DataFrame**) datu-egiturak.
- Datuen malgutasuna (falta diren datuak, denbora-serieak eta datu mota heterogeneoak kudeatzen ditu).
- Etiketatutako errenkadak eta zutabeak, datuak lerrokatzeko
- Indexazio malgua, *slicing*-a, *fancy indexing*-a eta datu multzo handietarako azpimultzo-hautaketa .

In [2]:
```
import numpy as np
import pandas as pd

pd.__version__
```

Out[2]: '2.2.3'

In [3]:
```
# sakatu TAB Pandas-en izen-espazioa ikusteko
#pd.
```

In [4]:
```
def pprint(*args, sep='\n', end='\n', userepr=True, align=False, breakline=False, inder
    "Ebaluatu eta modu txukunean inprimatu"
    if align :
        max_arg_len = max(map(len,args))
        txt2txt = lambda txt : ' '*(max_arg_len-len(txt)) + txt
        args = list(map(txt2txt, args))
    geval = lambda txt : eval(txt, globals())
    txt2txt = lambda txt : f'{repr(geval(txt))}' if userepr else f'{geval(txt)}'
    output = map(txt2txt, args)
    txt2txt = lambda txt : f'{txt} = '
    prefix = map(txt2txt, args)
    if breakline :
        po2txt = lambda p,o : (p+'\n'+o).replace('\n','\n'+indent)
    else:
        po2txt = lambda p,o : (p+o).replace('\n','\n'+' '*len(p))
    print(*map(po2txt, prefix, output), sep=sep, end=end)

class display(object):
    """Bistaratu hainbat objekturen HTML adierazpena"""
    template = """<div style="float: left; padding: 10px;">
    <p style='font-family:"Courier New", Courier, monospace'>{0}</p>{1}
    </div>"""
    def __init__(self, *args):
```

```
        self.args = args

    def _repr_html_(self):
        return '\n'.join(self.template.format(a, eval(a)._repr_html_())
                         for a in self.args)

    def __repr__(self):
        return '\n\n'.join(a + '\n' + repr(eval(a))
                           for a in self.args)

from IPython.display import HTML
HTML("""
<style>
.dataframe {
    font-size: 80% !important; /* Adjust the percentage as needed */
}
</style>
""")
```

Out[4]:

Pandas Series-ak

- Indexatutako dimentsio bakarreko arraya
- Bi atributu nagusi:
  - `values` : NumPy arraya
  - `index` : `pd.Index` motako array erako objektu bat

In [5]:
```
d = pd.Series([0.25, 0.5, 0.75, 1.0])
pprint('d', 'd.values', 'd.index', align=True)
```

```
      d = 0    0.25
          1    0.50
          2    0.75
          3    1.00
          dtype: float64
d.values = array([0.25, 0.5 , 0.75, 1.  ])
 d.index = RangeIndex(start=0, stop=4, step=1)
```

Series-ak NumPy arrayetatik sor daitezke:

In [6]:
```
d = pd.Series(np.linspace(0,4,6))
d
```

Out[6]:
```
0    0.0
1    0.8
2    1.6
3    2.4
4    3.2
5    4.0
dtype: float64
```

Series bat NumPy array bat bezala indexa daiteke:

In [7]:
```
d = pd.Series(np.arange(10,15))
pprint('d[3]')  # indize sinplea --> eskalarra
```

```
pprint('d[3:]', 'd[3:4]')  # zatia --> Series-a
pprint('d[[1,3,0]]', 'd[[1]]')  # indize aurreratua --> Series-a
```

```
d[3] = 13
d[3:] = 3    13
        4    14
        dtype: int64
d[3:4] = 3    13
         dtype: int64
d[[1,3,0]] = 1    11
             3    13
             0    10
             dtype: int64
d[[1]] = 1    11
         dtype: int64
```

Baina... Pandas Series-ek **indize esplizitu** bat izan dezakete

- Ematen ez bada, **osoko indize inplizitu** bat erabiltzen da `[0,1,...,n-1]`

In [8]:
```
d = pd.Series(np.arange(10,15), index=["a","b","c","d","e"])
pprint('d')
# ABISUA "treating keys as positions is deprecated"  --> use Series.iloc[pos]
#pprint('d[1]')
pprint('d["b"]')
```

```
d = a    10
    b    11
    c    12
    d    13
    e    14
    dtype: int64
d["b"] = 11
```

Slicing - 1. kasua - etiketetak indize esplizituan

- Slicean zenbaki osoak ez diren (esplizituak) nahiz osoak diren (inplizituak) balioak erabil daitezke.
- slicea ez-osoa → **etiketetan oinarritutako indexazioa** → $[from, to]$
- slicea osoa → **posizioan oinarritutako indexazioa** → $[from, to)$

In [9]:
```
d = pd.Series([100,101,102,103,104], index=["a","b","c","d","e"])
pprint('d')
pprint('d["a"]', 'd["c"]', 'd["a":"c"].values')   # indize esplizitua
pprint('d[0:2].values')                           # indize inplizitua
```

```
d = a    100
    b    101
    c    102
    d    103
    e    104
    dtype: int64
d["a"] = 100
d["c"] = 102
d["a":"c"].values = array([100, 101, 102])
d[0:2].values = array([100, 101])
```

Slicing - 2. kasua - zenbaki osoak indize esplizituan

- Slicing-a beti indize inplizituei dagokie.
- slicea osoa → **posizioan oinarritutako indexazioa** → $[from, to)$

In [10]:
```
d = pd.Series([100,101,102,103,104], index=[1,5,4,2,3])
pprint('d')
pprint('d[1]', 'd[3]')  # indize esplizitua
pprint('d[1:3].values') # indize inplizitua
```

```
d = 1    100
    5    101
    4    102
    2    103
    3    104
    dtype: int64
d[1] = 100
d[3] = 104
d[1:3].values = array([101, 102])
```

Fancy Indexing

- Etiketetan oinarritutako indexazioa
- Posizioan oinarritutako indexazioa zaharkituta dago

In [11]:
```
d = pd.Series([100,101,102,103,104], index=["a","b","c","d","e"])
pprint('d[["a","b","c"]].values')   # indize esplizitua
# ABISUA "treating keys as positions is deprecated"  --> use Series.iloc[pos]
#pprint('d[[1,2,3]].values')    # indize inplizitua

d = pd.Series([100,101,102,103,104], index=[1,5,4,2,3])
pprint('d[[1,2,3]].values')    # indize esplizitua
```

```
d[["a","b","c"]].values = array([100, 101, 102])
d[[1,2,3]].values = array([100, 103, 104])
```

Series-ak eta hiztegiak

- Series-ak Python-eko hiztegi espezializatu modukoak dira $\{index_{typed\ \&\ ordered} \rightarrow value_{typed}\}$
- `pd.Series(dict)` → hiztegi batetik Series bat sortu
- `pd.Series.to_dict()` → Series batetik hiztegi bat sortu

In [12]:
```
d1 = pd.Series([100,101,102], index=["a","b","c"])
x = d1.to_dict()
d2 = pd.Series(x)
pprint('d1','x','d2')
```

```
d1 = a    100
     b    101
     c    102
     dtype: int64
x = {'a': 100, 'b': 101, 'c': 102}
d2 = a    100
     b    101
     c    102
     dtype: int64
```

- `.keys()` → `.index`

In [13]:
```python
d = pd.Series([100,101,102,103,104], index=["a","b","c","d","e"])
pprint('type(d.index) == type(d.keys())')
pprint('d.index == d.keys()')
pprint('d.index is d.keys()')
```

```
type(d.index) == type(d.keys()) = True
d.index == d.keys() = array([ True,  True,  True,  True,  True])
d.index is d.keys() = True
```

- ∄ `.values()`

In [14]:
```python
d.values
# TypeError: 'numpy.ndarray' object is not callable
#d.values()
```

Out[14]:
```
array([100, 101, 102, 103, 104])
```

- `.items()` → `zip` objektua: `(index,value)`

In [15]:
```python
d = pd.Series([100,101,102,103,104], index=["a","b","c","d","e"])
pprint('d.items()')
print(*d.items())
```

```
d.items() = <zip object at 0x7f1fe4c39640>
('a', 100) ('b', 101) ('c', 102) ('d', 103) ('e', 104)
```

- `x in series` → Series-aren indizean `x` ote dagoen egiaztatzen

In [16]:
```python
d = pd.Series([100,101,102,103,104], index=["a","b","c","d","e"])
pprint('102 in d', '"xxx" in d')
```

```
102 in d = False
"xxx" in d = False
```

Pandas DataFrame-ak

- DataFrame-ak Python-eko hiztegi espezializatu modukoak dira $\{index_{typed\ \&\ ordered} \rightarrow series\}$ errenkada-indize komun batekin
- Errenkada-indize komuneko *Series-en Series* moduko bat
-
  - Hiru atributu nagusi:
  - `values` : NumPy arraya
  - `index` eta `columns` : `pd.Index` motako array gisako objektuak

In [18]:
```
cities = ['California','Texas','Florida','New York']
population = pd.Series([39538223,29145505,21538187,20201249], index=cities)
area = pd.Series([423967, 695662, 170312, 141297], index=cities)
states = pd.DataFrame({'population':population, 'area':area})
states
```

Out[18]:

	population	area
California	39538223	423967
Texas	29145505	695662
Florida	21538187	170312
New York	20201249	141297

- `index` atributua (*errenkada-indizea*) eta `columns` atributua (*zutabe-indizea*) `pd.Index` motakoak dira
- `values` atributuak (`np.ndarray` motakoa) datuak dauzka

In [19]:
```
states.index
```

Out[19]:
```
Index(['California', 'Texas', 'Florida', 'New York'], dtype='object')
```

In [20]:
```
states.columns
```

Out[20]:
```
Index(['population', 'area'], dtype='object')
```

In [21]:
```
states.values
```

Out[21]:
```
array([[39538223,   423967],
       [29145505,   695662],
       [21538187,   170312],
       [20201249,   141297]])
```

DataFrame bat Series bakar batetik eraiki daiteke:

In [22]:
```
cities = ['California','Texas','Florida','New York']
population = pd.Series([39538223,29145505,21538187,20201249], index=cities)
```

```
states = pd.DataFrame(population, columns=['population'])
states
```

Out[22]:

	population
California	39538223
Texas	29145505
Florida	21538187
New York	20201249

DataFrame bat Series hiztegi batetik eraiki daiteke:

In [23]:
```
cities = ['California','Texas','Florida','New York']
population = pd.Series([39538223,29145505,21538187,20201249], index=cities)
area = pd.Series([423967, 695662, 170312, 141297], index=cities)
states = pd.DataFrame({'population':population, 'area':area})
states
```

Out[23]:

	population	area
California	39538223	423967
Texas	29145505	695662
Florida	21538187	170312
New York	20201249	141297

DataFrame bat hiztegi zerrenda batetik eraiki daiteke:

In [24]:
```
data = [
    {'population':39538223, 'area':423967},
    {'population':29145505, 'area':695662},
    {'population':21538187, 'area':170312},
    {'population':20201249, 'area':141297}
]
states = pd.DataFrame(data, index=["California","Texas","Florida","New York"])
states
```

Out[24]:

	population	area
California	39538223	423967
Texas	29145505	695662
Florida	21538187	170312
New York	20201249	141297

DataFrame bat bi dimentsioko NumPy array batetik eraiki daiteke:

In [25]:
```
data = np.array([[39538223,   423967],
          [29145505,   695662],
          [21538187,   170312],
          [20201249,   141297]])
```

```
states = pd.DataFrame(data,
                      columns=["population","area"],
                      index=["California","Texas","Florida","New York"])
states
```

Out[25]:

	population	area
California	39538223	423967
Texas	29145505	695662
Florida	21538187	170312
New York	20201249	141297

DataFrame batek orden desberdineko indizeak integra ditzake

In [26]:
```
cities1 = ['California','Texas','Florida','New York']
population = pd.Series([39538223,29145505,21538187,20201249], index=cities1)
cities2 = ['Texas','Florida','New York','California']
area = pd.Series([695662, 170312, 141297, 423967], index=cities2)
states = pd.DataFrame({'population':population, 'area':area})
states
```

Out[26]:

	population	area
California	39538223	423967
Florida	21538187	170312
New York	20201249	141297
Texas	29145505	695662

DataFrame batek balio desberdineko indizeak integra ditzake

- *Falta diren* balioak `NaN` balioekin betetzen dira

In [27]:
```
cities1 = ['California','Texas','Florida']
population = pd.Series([39538223,29145505,21538187], index=cities1)
cities2 = ['Texas','Florida','New York']
area = pd.Series([695662, 170312, 141297], index=cities2)
states = pd.DataFrame({'population':population, 'area':area})
states
```

Out[27]:

	population	area
California	39538223.0	NaN
Florida	21538187.0	170312.0
New York	NaN	141297.0
Texas	29145505.0	695662.0

Pandas Index-a

- NumPy-ko array aldaezin moduko bat

- NumPy-ko arrayen atributu asko ditu; indexatu, slicing eta abar egin daiteke.

In [28]:
```
ind = pd.Index(['California', 'Texas', 'Florida'])
pprint('ind', 'ind[1]', 'ind[:2]', 'ind[[2,0,1]]', align=True, end='\n\n')
pprint('ind.size', 'ind.shape', 'ind.ndim', 'ind.dtype', align=True)
```

```
         ind = Index(['California', 'Texas', 'Florida'], dtype='object')
      ind[1] = 'Texas'
     ind[:2] = Index(['California', 'Texas'], dtype='object')
ind[[2,0,1]] = Index(['Florida', 'California', 'Texas'], dtype='object')

 ind.size = 3
ind.shape = (3,)
 ind.ndim = 1
ind.dtype = dtype('O')
```

In [29]:
```
ind = pd.Index([200, 1000, 300])
pprint('ind', 'ind[1]', 'ind[:2]', 'ind[[2,0,1]]', align=True, end='\n\n')
pprint('ind.size', 'ind.shape', 'ind.ndim', 'ind.dtype', align=True)
```

```
         ind = Index([200, 1000, 300], dtype='int64')
      ind[1] = 1000
     ind[:2] = Index([200, 1000], dtype='int64')
ind[[2,0,1]] = Index([300, 200, 1000], dtype='int64')

 ind.size = 3
ind.shape = (3,)
 ind.ndim = 1
ind.dtype = dtype('int64')
```

`pd.Index` -k Python-eko `set` objektuen konbentzio asko jarraitzen ditu:

- `idx1.union(idx2)` → bi Index-etako elementuak konbinatzen ditu
- `idx1.ebakidura(idx2)` → bi Index-etan komunak diren elementuak
- `idx1.difference(idx2)` → `idx1` -en egon eta `idx2` -n ez dauden elementuak
- `idx1.symmetric_difference(idx2)` → soilik bietako batean dauden elementuak
- `idx1.equals(idx2)` → elementu berak eta orden berean dituzten egiaztatzen du

In [30]:
```
idx1 = pd.Index([1,2,3,4,5])
idx2 = pd.Index([4,5,6,7])
pprint('idx1.union(idx2)', 'idx1.intersection(idx2)',
       'idx1.difference(idx2)', 'idx1.symmetric_difference(idx2)',
       'idx1.equals(idx2)', align=True)
```

```
             idx1.union(idx2) = Index([1, 2, 3, 4, 5, 6, 7], dtype='int64')
      idx1.intersection(idx2) = Index([4, 5], dtype='int64')
        idx1.difference(idx2) = Index([1, 2, 3], dtype='int64')
idx1.symmetric_difference(idx2) = Index([1, 2, 3, 6, 7], dtype='int64')
            idx1.equals(idx2) = False
```

Pandas-en Sarrera/Irteera

- S/I funtzio-bilduma ahaltsu eta malgua
  - Testu-fitxategiak: CSV, TSV, testu arrunta

- Kalkulu-orriak: Excel, OpenOffice/LibreOffice
    - Webeko datuak: HTML
    - Datu-formatu egituratuak: JSON, XML
    - Datu-base erlazionalak: SQL
    - Beste formatu batzuk: Pickle, HDF5, Parquet, ...
- Sarrera (`pd.read_...`): `pd.read_csv` , `pd.read_excel` , `pd.read_json` ,... `
- Irteera (`df.to_...`): `df.to_csv` , `df.to_excel` , `df.to_json` ,... `

In [31]:
```
#pd.read_
#df.to_
```

In [34]:
```
url = "https://raw.githubusercontent.com/pandas-dev/pandas/refs/heads/main/pandas/tests
pd.read_csv(url)
```

Out[34]:

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

In [33]:
```
#!pip install openpyxl
url = "https://github.com/pandas-dev/pandas/raw/refs/heads/main/pandas/tests/io/data/ex
pd.read_excel(url, index_col=0) # erabili lehen zutabea indize gisa
```

Out[33]:

	A	B	C	D
2000-01-03	0.980269	3.685731	-0.364217	-1.159738
2000-01-04	1.047916	-0.041232	-0.161812	0.212549
2000-01-05	0.498581	0.731168	-0.537677	1.346270
2000-01-06	1.120202	1.567621	0.003641	0.675253
2000-01-07	-0.487094	0.571455	-1.611639	0.103469
2000-01-10	0.836649	0.246462	0.588543	1.062782
2000-01-11	-0.157161	1.340307	1.195778	-1.097007

Indexazioa eta hautaketa

In [35]:
```python
cities = ['California', 'Texas', 'Florida', 'New York', 'Pennsylvania']
area = pd.Series([423967, 695662, 170312, 141297, 119280], index=cities)
population = pd.Series([39538223, 29145505, 21538187, 20201249, 13002700], index=cities
gdp = pd.Series([3.9, 2.7, 1.3, 1.8, 1.0], index=cities) # Barne Produktu Gordina
df = pd.DataFrame({'area':area, 'population':population, 'GDP':gdp})
df
```

Out[35]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

- Indexazio sinplea → zutabe-hautaketa (**Series**)

In [36]:
```python
df['area']
# KeyError: 0
#df[0]
```

Out[36]:
```
California      423967
Texas           695662
Florida         170312
New York        141297
Pennsylvania    119280
Name: area, dtype: int64
```

- Fancy Indexing → zutabeen hautaketa (**DataFrame**)

In [37]:
```python
display( "df[['GDP','area']]", "df[['area']]")
```

Out[37]:

df[['GDP','area']]

	GDP	area
California	3.9	423967
Texas	2.7	695662
Florida	1.3	170312
New York	1.8	141297
Pennsylvania	1.0	119280

df[['area']]

	area
California	423967
Texas	695662
Florida	170312
New York	141297
Pennsylvania	119280

In [38]:
```python
pprint("df['GDP']")
```

```
df['GDP'] = California      3.9
            Texas           2.7
            Florida         1.3
            New York        1.8
            Pennsylvania    1.0
            Name: GDP, dtype: float64
```

- Slicing → errenkaden hautaketa (**DataFrame**)

In [39]:
```
display('df[:2]', 'df[1:2]', 'df[2:2]')
```

Out[39]:

df[:2]

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7

df[1:2]

	area	population	GDP
Texas	695662	29145505	2.7

df[2:2]

	area	population	GDP

Zutabe eta errenkaden hautaketen konbinaketa

- zutabeak fancy indexing-arekin hautatzen dira
- errenkadak (posizioaren araberako) slicing-arekin hautatzen dira

In [40]:
```
display("df[['area','population']][:3]" , "df[:3][['area','population']]")
```

Out[40]:

df[['area','population']][:3]

	area	population
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

df[:3][['area','population']]

	area	population
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

Datuen atzipena `df.values` bidez

- `df.values` → NumPy array homogeneoa
- Series-en datu motak goragoko mota batera bihur daitezke (*upcasting*)

In [41]:
```
pprint('df.values')
pprint('df.values.dtype', 'df["area"].dtype',
       'df["population"].dtype', 'df["GDP"].dtype', align=True)
```

```
df.values = array([[4.2396700e+05, 3.9538223e+07, 3.9000000e+00],
                   [6.9566200e+05, 2.9145505e+07, 2.7000000e+00],
                   [1.7031200e+05, 2.1538187e+07, 1.3000000e+00],
                   [1.4129700e+05, 2.0201249e+07, 1.8000000e+00],
                   [1.1928000e+05, 1.3002700e+07, 1.0000000e+00]])
       df.values.dtype = dtype('float64')
      df["area"].dtype = dtype('int64')
df["population"].dtype = dtype('int64')
       df["GDP"].dtype = dtype('float64')
```

Konparazio-eragilearekin maskaratzea

- Izan bedi `op` eragile bat
- `Series op value` → broadcasting eragiketa → Series
- `DataFrame op value` → broadcasting eragiketa → DataFrame

In [42]:
```
pprint("df['GDP'] > 2")
```

```
df['GDP'] > 2 = California       True
                Texas            True
                Florida         False
                New York        False
                Pennsylvania    False
                Name: GDP, dtype: bool
```

In [43]:
```
display("df * 10" , "df > 2")
```

Out[43]:

df * 10

	area	population	GDP
California	4239670	395382230	39.0
Texas	6956620	291455050	27.0
Florida	1703120	215381870	13.0
New York	1412970	202012490	18.0
Pennsylvania	1192800	130027000	10.0

df > 2

	area	population	GDP
California	True	True	True
Texas	True	True	True
Florida	True	True	False
New York	True	True	False
Pennsylvania	True	True	False

Series/DataFrame boolearrak indize gisa erabil daitezke

- Series boolearra → errenkadak hautatu
- DataFrame boolearra → elementuak hautatu (eta `NaN` balioekin bete)

In [44]:
```
display("df[df['GDP'] > 2]" , "df[df > 2]")
```

Out[44]:

df[df['GDP'] > 2]

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7

df[df > 2]

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	NaN
New York	141297	20201249	NaN
Pennsylvania	119280	13002700	NaN

`GDP>1` eta `population<30000000` betetzen duten hirien azalera: :

In [45]:
```
df[df['GDP'] > 1]
```

Out[45]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8

In [46]:
```
df[(df['GDP']>1) & (df['population']<30000000)]
```

Out[46]:

	area	population	GDP
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8

In [47]:
```
df[(df['GDP']>1) & (df['population']<30000000)]['area']
```

Out[47]:
```
Texas       695662
Florida     170312
New York    141297
Name: area, dtype: int64
```

Indexatzaileak: `.loc` and `.iloc`

- `.loc` → indexazio (eta slicing) esplizitua
- `.iloc` → indexazio (eta slicing) inplizitua
- NumPy moduko indexazioa
  - `.loc[i]` → `i` errenkada
  - `.loc[i,j]` → `i` errenkada, `j` zutabea
  - Indexazio sinplea, fancy indexing and slicing

In [48]:
```
df
```

Out[48]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

In [49]:
```
df.loc['Florida']
```

Out[49]:
```
area            170312.0
population    21538187.0
GDP                  1.3
Name: Florida, dtype: float64
```

In [50]:
```
df.loc[['Florida','Texas']]
```

Out[50]:

	area	population	GDP
Florida	170312	21538187	1.3
Texas	695662	29145505	2.7

In [51]:
```
df.loc[:'Florida']
```

Out[51]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3

In [52]:
```
df.iloc[2]
```

Out[52]:
```
area            170312.0
population    21538187.0
GDP                  1.3
Name: Florida, dtype: float64
```

In [53]:
```
df.iloc[[2,1]]
```

Out[53]:

	area	population	GDP
Florida	170312	21538187	1.3
Texas	695662	29145505	2.7

In [54]:
```
df.iloc[:3]
```

Out[54]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3

In [55]:
```
df.loc['Florida','population']
```

Out[55]:
```
21538187
```

In [56]:
```
df.loc[['California','Florida'],'population']
```

Out[56]:
```
California    39538223
Florida       21538187
Name: population, dtype: int64
```

In [57]:
```
df.loc[['California','Florida'],:]
```

Out[57]:

	area	population	GDP
California	423967	39538223	3.9
Florida	170312	21538187	1.3

`.iloc` erabiltzea `df.values` erabiltzearen antzekoa da, baina DataFrame-aren egitura mantentzen du (NumPy array homogeneoaren aldean)

In [58]:
```
df.iloc[:2,[0,2]]
```

Out[58]:

	area	GDP
California	423967	3.9
Texas	695662	2.7

In [59]:
```
pprint("df.values[:2,[0,2]]" , "df.values[:2,[0,2]].dtype", align=True)
```

```
      df.values[:2,[0,2]] = array([[4.23967e+05, 3.90000e+00],
                                   [6.95662e+05, 2.70000e+00]])
df.values[:2,[0,2]].dtype = dtype('float64')
```

DataFrame-ak indexazio bidez aldatzen

- Indexazioa DataFrame bat aldatzeko erabil daiteke
  - Balioak aldatu
  - Zutabeak gehitu/kendu
  - Errenkadak gehitu/kendu

DataFrame-eko balioak aldatzen

- Indexazio sinplea → zutabe-esleipena

In [60]:
```
df2 = df.copy()
df2['area'] = 0
df2
```

Out[60]:

	area	population	GDP
California	0	39538223	3.9
Texas	0	29145505	2.7
Florida	0	21538187	1.3
New York	0	20201249	1.8
Pennsylvania	0	13002700	1.0

- Fancy Indexing → zutabeen esleipena

In [61]:
```
df2 = df.copy()
df2[['area','GDP']] = 0
df2
```

Out[61]:

	area	population	GDP
California	0	39538223	0
Texas	0	29145505	0
Florida	0	21538187	0
New York	0	20201249	0
Pennsylvania	0	13002700	0

- Slicing → errenkaden esleipena

In [62]:
```
df2 = df.copy()
df2[:2] = 0
df2
```

Out[62]:

	area	population	GDP
California	0	0	0.0
Texas	0	0	0.0
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

- `df.values[i,j]` -ri esleitzeak ez du DataFrame-a aldatzen

In [63]:
```
df2 = df.copy()
df2.values[:2,:2] = 0
df2
```

Out[63]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

- Series bidezko maskaratzea (indizea Series boolear bat da) → errenkaden esleipena

In [64]:
```
df2 = df.copy()
pprint('df2["GDP"]>2')
df2[df2['GDP']>2] = 0
df2
```

```
df2["GDP"]>2 = California       True
               Texas            True
               Florida         False
               New York        False
               Pennsylvania    False
               Name: GDP, dtype: bool
```

Out[64]:

	area	population	GDP
California	0	0	0.0
Texas	0	0	0.0
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

- DataFrame bidezko maskaratzea (indizea DataFrame boolear bat da) → *elementuen* esleipena

In [65]:
```
df2 = df.copy()
pprint('df2>2')
df2[df2>2] = 0
df2
```

```
df2>2 =                 area  population    GDP
        California      True        True   True
        Texas           True        True   True
        Florida         True        True  False
        New York        True        True  False
        Pennsylvania    True        True  False
```

Out[65]:

	area	population	GDP
California	0	0	0.0
Texas	0	0	0.0
Florida	0	0	1.3
New York	0	0	1.8
Pennsylvania	0	0	1.0

- `.loc` and `.iloc` → *azpimultzoen* esleipena

In [66]:
```
df2 = df.copy()
df2.loc[['California','New York'],'population':] = 0
df2
```

Out[66]:

	area	population	GDP
California	423967	0	0.0
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	0	0.0
Pennsylvania	119280	13002700	1.0

In [67]:
```
df2 = df.copy()
df2.iloc[[0,3],1:] = 0
df2
```

Out[67]:

	area	population	GDP
California	423967	0	0.0
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	0	0.0
Pennsylvania	119280	13002700	1.0

Zutabeak gehitzen/kentzen

- **Gehitu**: `df[new_index] = value`
- **Kendu**: `del df[index]` edo `df.drop(index, axis=1, inplace=True)`

In [68]:
```
# gehitu 'density' zutabea
df2 = df.copy()
df2['density'] = df2['population'] / df2['area']
df2
```

Out[68]:

	area	population	GDP	density
California	423967	39538223	3.9	93.257784
Texas	695662	29145505	2.7	41.896072
Florida	170312	21538187	1.3	126.463121
New York	141297	20201249	1.8	142.970120
Pennsylvania	119280	13002700	1.0	109.009893

In [69]:
```
# kendu zutabea 'population'
df2 = df.copy()
del df2['population']
df2
```

Out[69]:

	area	GDP
California	423967	3.9
Texas	695662	2.7
Florida	170312	1.3
New York	141297	1.8
Pennsylvania	119280	1.0

In [70]:
```
# kendu 'population' zutabea
df2 = df.copy()
df2.drop('population', axis=1, inplace=True)
df2
```

Out[70]:

	area	GDP
California	423967	3.9
Texas	695662	2.7
Florida	170312	1.3
New York	141297	1.8
Pennsylvania	119280	1.0

In [71]:

```
# kendu ['area','population'] zutabeak
df2 = df.copy()
df2.drop(['area','population'], axis=1, inplace=True)
df2
```

Out[71]:

	GDP
California	3.9
Texas	2.7
Florida	1.3
New York	1.8
Pennsylvania	1.0

Errenkadak gehitzen/kentzen

- **Gehitu**: `df.loc[new_index] = value` (WARNING → homogeneous DataFrame)
- **Kendu**: `df.drop(index, axis=0, inplace=True)`

In [72]:

```
# gehitu 'chicago' errenkada
df2 = df.copy()
df2.loc['Chicago'] = [111111,22222222,2.0]
df2
```

Out[72]:

	area	population	GDP
California	423967.0	39538223.0	3.9
Texas	695662.0	29145505.0	2.7
Florida	170312.0	21538187.0	1.3
New York	141297.0	20201249.0	1.8
Pennsylvania	119280.0	13002700.0	1.0
Chicago	111111.0	22222222.0	2.0

In [73]:

```
# kendu 'New York' errenkada
df2 = df.copy()
df2.drop('New York', axis=0, inplace=True)
df2
```

Out[73]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
Pennsylvania	119280	13002700	1.0

In [74]:

```
# kendu ['California','New York'] errenkadak
df2 = df.copy()
df2.drop(['California','New York'], axis=0, inplace=True)
df2
```

Out[74]:

	area	population	GDP
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
Pennsylvania	119280	13002700	1.0

Datuekin eragiketak egiten

- Eragile aritmetikoak eta NumPy funtzioak Series/DataFrame-ekin erabil daitezke
- Indizeak mantendu egiten dira
- Eragiketa/funtzio bitarrek indizeak lerrokatzen dituzte
    - Datu osatugabeak kudea ditzakete `NaN` balioak txertatuz
- DataFrame eta Series arteko eragiketek broadcasting propietatea dute

Indizeen mantentzea

In [75]:

```
df
```

Out[75]:

	area	population	GDP
California	423967	39538223	3.9
Texas	695662	29145505	2.7
Florida	170312	21538187	1.3
New York	141297	20201249	1.8
Pennsylvania	119280	13002700	1.0

In [76]:

```
df * 10
```

Out[76]:

	area	population	GDP
California	4239670	395382230	39.0
Texas	6956620	291455050	27.0
Florida	1703120	215381870	13.0
New York	1412970	202012490	18.0
Pennsylvania	1192800	130027000	10.0

In [77]:
```python
np.log(df[['area','population']])
```

Out[77]:

	area	population
California	12.957411	17.492778
Texas	13.452619	17.187811
Florida	12.045387	16.885338
New York	11.858619	16.821255
Pennsylvania	11.689229	16.380668

Python-eko eragileek metodo baliokideak dituzte Pandas objektuetan:

Python-eko eragilea	Pandas metodoa(k)
`+`	`add`
`-`	`sub` , `subtract`
`*`	`mul` , `multiply`
`/`	`truediv` , `div` , `divide`
`//`	`floordiv`
`%`	`mod`
`**`	`pow`

Indizeen lerrokatzea

In [78]:
```python
x = pd.Series({'A': 10,'B': 20})
y = pd.Series({'B': 4, 'A': 5})
x / y
```

Out[78]:
```
A    2.0
B    5.0
dtype: float64
```

In [79]:
```python
x = pd.Series({'A': 10,'B': 20, 'C':30})
y = pd.Series({'B': 4, 'C': 5, 'D':3})
x / y
```

Out[79]:
```
A    NaN
B    5.0
C    6.0
D    NaN
dtype: float64
```

In [80]:
```python
df1 = pd.DataFrame({
    'first':pd.Series([1,2],index=["A","B"]),
    'second':pd.Series([1,2],index=["A","B"])})
df2 = pd.DataFrame({
    'first':pd.Series([3,2],index=["A","C"]),
    'second':pd.Series([2,4],index=["A","C"])})
```

In [81]:
```python
display("df1", "df2" , "df1 + df2")
```

Out[81]: df1

	first	second
A	1	1
B	2	2

df2

	first	second
A	3	2
C	2	4

df1 + df2

	first	second
A	4.0	3.0
B	NaN	NaN
C	NaN	NaN

Pandas-en funtzio aritmetikoek `fill_value` parametroa dute falta diren sarreren ordez erabiltzeko:

In [82]:
```
df1.add(df2)
```

Out[82]:

	first	second
A	4.0	3.0
B	NaN	NaN
C	NaN	NaN

In [83]:
```
df1.add(df2, fill_value=0)
```

Out[83]:

	first	second
A	4.0	3.0
B	2.0	2.0
C	2.0	4.0

In [84]:
```
df1.loc['C']=0
df2.loc['B']=0
display("df1" , "df2" , "df1+df2")
```

Out[84]: df1

	first	second
A	1	1
B	2	2
C	0	0

df2

	first	second
A	3	2
C	2	4
B	0	0

df1+df2

	first	second
A	4	3
B	2	2
C	2	4

DataFrame-en eta Series-en arteko eragiketak

- Broadcasting-a, NumPy-ren *antzekoa*
    - Errenkada-eragiketetara mugatua

In [85]:
```
ints = np.random.randint(0,10,(3,4))
df = pd.DataFrame(ints, columns=['A','B','C','D'], index=['x','y','z'])
df
```

Out[85]:

	A	B	C	D
x	1	5	0	4
y	8	3	4	5
z	3	6	1	9

In [86]:
```python
display("df" , "df - df.loc['x']" , "df.sub(df.loc['x'], axis=1)")
```

Out[86]:

df

	A	B	C	D
x	1	5	0	4
y	8	3	4	5
z	3	6	1	9

df - df.loc['x']

	A	B	C	D
x	0	0	0	0
y	7	-2	4	1
z	2	1	1	5

df.sub(df.loc['x'], axis=1)

	A	B	C	D
x	0	0	0	0
y	7	-2	4	1
z	2	1	1	5

In [87]:
```python
df
```

Out[87]:

	A	B	C	D
x	1	5	0	4
y	8	3	4	5
z	3	6	1	9

In [88]:
```python
# Broadcasting-aren arauak...
display("df - df['A']" , "df.sub(df['A'], axis=0)")
```

Out[88]:

df - df['A']

	A	B	C	D	x	y	z
x	NaN	NaN	NaN	NaN	NaN	NaN	NaN
y	NaN	NaN	NaN	NaN	NaN	NaN	NaN
z	NaN	NaN	NaN	NaN	NaN	NaN	NaN

df.sub(df['A'], axis=0)

	A	B	C	D
x	0	4	-1	3
y	0	-5	-4	-3
z	0	3	-2	6

`pd.DataFrame(df['A'])` zutabe bakarreko DataFrame bat da, baina ez dago broadcasting-ik DataFrame-en artean...

In [89]:
```python
display("pd.DataFrame(df['A'])" , "df - pd.DataFrame(df['A'])")
```

Out[89]:

pd.DataFrame(df['A'])

	A
x	1
y	8
z	3

df - pd.DataFrame(df['A'])

	A	B	C	D
x	0	NaN	NaN	NaN
y	0	NaN	NaN	NaN
z	0	NaN	NaN	NaN

Falta diren datuen kudeaketa

Pandas-ek *balio jagoleak* (*Sentinel Values*) erabiltzen ditu erabilgarri ez dauden (NA) balioak kudeatzeko:

- `None`, `np.nan` and `pd.NA`
  - Zenbaki osoak: `pd.NA`
  - Zenbaki errealak: `np.nan` and `pd.NA`
  - Objektu orokorrak: `None`, `np.nan` and `pd.NA`
- Defektuz, zenbakizko datuetan, `None` → `np.nan` (floating point)

In [90]:
```
s = pd.Series([None,2,3])
pprint('s', 's[0]', 'type(s[0])', align=True)
```

```
        s = 0    NaN
            1    2.0
            2    3.0
            dtype: float64
     s[0] = nan
type(s[0]) = <class 'numpy.float64'>
```

In [91]:
```
s = pd.Series([np.nan,2,3])
pprint('s', 's[0]', 'type(s[0])', align=True)
```

```
        s = 0    NaN
            1    2.0
            2    3.0
            dtype: float64
     s[0] = nan
type(s[0]) = <class 'numpy.float64'>
```

In [92]:
```
s = pd.Series([pd.NA,2,3])
pprint('s', 's[0]', 'type(s[0])', align=True)
```

```
        s = 0    <NA>
            1       2
            2       3
            dtype: object
     s[0] = <NA>
type(s[0]) = <class 'pandas._libs.missing.NAType'>
```

In [93]:
```
s = pd.Series([None,2,3], dtype='Int64')
pprint('s', 's[0]', 'type(s[0])', align=True)
```

```
         s = 0    <NA>
             1       2
             2       3
             dtype: Int64
      s[0] = <NA>
type(s[0]) = <class 'pandas._libs.missing.NAType'>
```

In [94]:
```
s = pd.Series([np.nan,2,3], dtype='Int64')
pprint('s', 's[0]', 'type(s[0])', align=True)
```

```
         s = 0    <NA>
             1       2
             2       3
             dtype: Int64
      s[0] = <NA>
type(s[0]) = <class 'pandas._libs.missing.NAType'>
```

In [95]:
```
pprint('pd.Series([None,"a",123])')
```

```
pd.Series([None,"a",123]) = 0    None
                            1       a
                            2     123
                            dtype: object
```

In [96]:
```
pprint('pd.Series([np.nan,"a",123])')
```

```
pd.Series([np.nan,"a",123]) = 0    NaN
                              1      a
                              2    123
                              dtype: object
```

In [97]:
```
pprint('pd.Series([pd.NA,"a",123])')
```

```
pd.Series([pd.NA,"a",123]) = 0    <NA>
                             1       a
                             2     123
                             dtype: object
```

Pandas-en goranzko mota-bihurketaren konbentzioak, NA balioak sartzen direnean:

Mota-klasea	NA gordetzean egindako bihurketa	NA balio jagolea
`floating`	Aldaketarik ez	`np.nan`
`object`	Aldaketarik ez	`None` edo `np.nan`
`integer`	`float64` bihurtu	`np.nan`
`boolean`	`object` bihurtu	`None` edo `np.nan`

Balio nuluak baztertzen

- NumPy-ko funtzio gehienek ez dituzte `nan` balioak kontuan hartzen (`nan` sortzen dute)
  - `nan` balioak kontuan hartzen dituzten NumPy funtzioak (`nan` balioak baztertzen dituzte):
    - `np.nansum`
    - `np.nanmin`
    - `np.nanmax`
  - NumPy-k ez du `pd.NA` kudeatzen → **ERROR**

- Pandas funtzioek `NaN` eta `NA` kontuan hartzen dituzte (`nan` balioak baztertzen dituzte)

In [98]:
```python
s = pd.Series([1,2,None,3,4])
pprint('s.values',
       's.values.sum()', 's.values.min()', 's.values.max()',
       'np.sum(s.values)', 'np.min(s.values)', 'np.max(s.values)',
       'np.nansum(s.values)', 'np.nanmin(s.values)', 'np.nanmax(s.values)',
       's.sum()', 's.min()', 's.max()', align=True)
```

```
          s.values = array([ 1.,  2., nan,  3.,  4.])
    s.values.sum() = nan
    s.values.min() = nan
    s.values.max() = nan
  np.sum(s.values) = nan
  np.min(s.values) = nan
  np.max(s.values) = nan
np.nansum(s.values) = 10.0
np.nanmin(s.values) = 1.0
np.nanmax(s.values) = 4.0
           s.sum() = 10.0
           s.min() = 1.0
           s.max() = 4.0
```

In [99]:
```python
s = pd.Series([1,2,pd.NA,3,4])
pprint('s.values',
       's.sum()', 's.min()', 's.max()', align=True)
# ERROREA!
#pprint('s.values.sum()', 's.values.min()', 's.values.max()',
#        'np.sum(s.values)', 'np.min(s.values)', 'np.max(s.values)',
#        'np.nansum(s.values)', 'np.nanmin(s.values)', 'np.nanmax(s.values)')
```

```
s.values = array([1, 2, <NA>, 3, 4], dtype=object)
 s.sum() = 10
 s.min() = 1
 s.max() = 4
```

Balio nuluak detektatzen

- `isnull` → (`True` / `False`) maskara boolearra balio nuluak dauden tokietan
- `notnull` → `isnull` -en aurkakoa

In [100…
```python
s = pd.Series([1,2,None,3,4])
pprint('s.isnull()', 's.notnull()', sep="\n\n", align=True)
```

```
s.isnull() = 0    False
             1    False
             2     True
             3    False
             4    False
             dtype: bool

s.notnull() = 0     True
              1     True
              2    False
              3     True
              4     True
              dtype: bool
```

In [101…
```
s[s.notnull()]
```

Out[101…
```
0    1.0
1    2.0
3    3.0
4    4.0
dtype: float64
```

Balio nuluak kentzen

- `dropna` → balio nuluak kenduta dituen kopia
  - Series: balio nuluak kendu
  - DataFrame: balio nuluak dituzten errenkadak/zutabeak kendu
- `inplace` parametroa: bool, defektuz `False`
  - `True` bada, lekuan bertan kentzen/betetzen du. Oharra: objektu honen beste bistak ere aldatuko ditu (adibidez, DataFrame bateko zutabe baten kopiarik gabeko zatia).

In [102…
```
s = pd.Series([1,2,None,3,4])
s.dropna()
```

Out[102…
```
0    1.0
1    2.0
3    3.0
4    4.0
dtype: float64
```

In [103…
```
pprint('s',end='\n\n')
s.dropna(inplace=True)
pprint('s')
```

```
s = 0    1.0
    1    2.0
    2    NaN
    3    3.0
    4    4.0
    dtype: float64

s = 0    1.0
    1    2.0
    3    3.0
    4    4.0
    dtype: float64
```

In [104…
```
a = np.arange(15,dtype='float64').reshape(3,5)
a[(0,2,2),(3,0,2)] = np.nan
df = pd.DataFrame(a)
df
```

Out[104…

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	NaN	13.0	14.0

In [105…
```
df.dropna() # df.dropna(axis=0)
```

Out[105…

	0	1	2	3	4
1	5.0	6.0	7.0	8.0	9.0

In [106…
```
df.dropna(axis=1)
```

Out[106…

	1	4
0	1.0	4.0
1	6.0	9.0
2	11.0	14.0

Balio nuluak betetzen

- `fillna` → balio nuluak ordezkatuta dituen kopia
- `inplace` parametroa: bool, lehenetsia False
  - `True` bada, lekuan bertan kentzen/betetzen du. Oharra: objektu honen beste bistak ere aldatuko ditu (adibidez, DataFrame bateko zutabe baten kopiarik gabeko zatia).
- **ZAHARKITUTAKO** `method` parametroa: { `bfill` , `ffill` , `None` }, defektuz `None`
  - `ffill` → *aurrerantz betetzea*, aurreko balioa aurrerantz hedatu
  - `bfill` → *atzerantz betetzea*, hurrengo balioa atzerantz hedatu
- `data.ffill()` eta `data.bfill()` erabili honen ordez:
  - `axis` parametroa: aurrerantz/atzerantz betetzeko ardatza

In [107…
```
s = pd.Series([1, None, 2, None, 3], index=list('abcde'), dtype='Int64')
pprint('s', 's.fillna(0)', 's.ffill()', 's.bfill()', align=True)
```

```
          s = a       1
              b    <NA>
              c       2
              d    <NA>
              e       3
              dtype: Int64
s.fillna(0) = a    1
              b    0
              c    2
              d    0
              e    3
              dtype: Int64
  s.ffill() = a    1
              b    1
              c    2
              d    2
              e    3
              dtype: Int64
  s.bfill() = a    1
              b    2
              c    2
              d    3
              e    3
              dtype: Int64
```

In [108…

```
a = np.arange(15,dtype='float64').reshape(3,5)
a[(0,2,2),(3,0,2)] = np.nan
df = pd.DataFrame(a)

display("df" , "df.fillna(0)")
```

Out[108…

df

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	NaN	13.0	14.0

df.fillna(0)

	0	1	2	3	4
0	0.0	1.0	2.0	0.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	0.0	11.0	0.0	13.0	14.0

In [109…

```
df
```

Out[109…

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	NaN	13.0	14.0

In [110…

```
display("df.ffill()" , "df.ffill(axis=1)")
```

Out[110…

df.ffill()

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	4.0
1	5.0	6.0	7.0	8.0	9.0
2	5.0	11.0	7.0	13.0	14.0

df.ffill(axis=1)

	0	1	2	3	4
0	0.0	1.0	2.0	2.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	11.0	13.0	14.0

In [111…

```
df
```

Out[111…

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	NaN	13.0	14.0

In [112…

```
display("df.bfill()" , "df.bfill(axis=1)")
```

Out[112…

df.bfill()

	0	1	2	3	4
0	0.0	1.0	2.0	8.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	NaN	11.0	NaN	13.0	14.0

df.bfill(axis=1)

	0	1	2	3	4
0	0.0	1.0	2.0	4.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	11.0	11.0	13.0	13.0	14.0

Datu multzoak kateatzen

In [113…

```
def make_df(col_index,row_index):
    """DataFrame bat azkar sortu"""
    data = {c: [str(c) + str(r) for r in row_index] for c in col_index}
    return pd.DataFrame(data, index=list(row_index), columns=list(col_index))

# DataFrame adibidea
display("make_df('ABCDEF',range(3))" , "make_df('ABCDEF','xyz')")
```

Out[113…

make_df('ABCDEF',range(3))

	A	B	C	D	E	F
0	A0	B0	C0	D0	E0	F0
1	A1	B1	C1	D1	E1	F1
2	A2	B2	C2	D2	E2	F2

make_df('ABCDEF','xyz')

	A	B	C	D	E	F
x	Ax	Bx	Cx	Dx	Ex	Fx
y	Ay	By	Cy	Dy	Ey	Fy
z	Az	Bz	Cz	Dz	Ez	Fz

`pd.concat()`

- NumPy-ko `concatenate` -ren antzekoa

```
pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None,
          levels=None, names=None, verify_integrity=False,
          sort=False, copy=True)
```

In [114…

```
s1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
s2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
pd.concat([s1, s2])
```

Out[114…

```
1    A
2    B
3    C
4    D
5    E
6    F
dtype: object
```

In [115…

```
s1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
s2 = pd.Series(['D', 'E', 'F'], index=[1, 2, 3])
pd.concat([s1, s2], axis=1)
```

Out[115…

	0	1
1	A	D
2	B	E
3	C	F

In [116…

```
df1 = make_df('AB', 'uv')
df2 = make_df('AB', 'xy')
display('df1', 'df2', 'pd.concat([df1, df2])')
```

Out[116…

df1

	A	B
u	Au	Bu
v	Av	Bv

df2

	A	B
x	Ax	Bx
y	Ay	By

pd.concat([df1, df2])

	A	B
u	Au	Bu
v	Av	Bv
x	Ax	Bx
y	Ay	By

In [117…

```
df1 = make_df('AB', 'xy')
df2 = make_df('CD', 'xy')
display('df1', 'df2', 'pd.concat([df1, df2], axis=1)')
```

Out[117…

df1

	A	B
x	Ax	Bx
y	Ay	By

df2

	C	D
x	Cx	Dx
y	Cy	Dy

pd.concat([df1, df2], axis=1)

	A	B	C	D
x	Ax	Bx	Cx	Dx
y	Ay	By	Cy	Dy

- **Indizeen mantentzea**: `pd.concat` -ek indizea *mantentzen* du (emaitzak indize bikoiztuak izan ditzake)

In [118…

```
df1 = make_df('AB', 'xy')
df2 = make_df('AB', 'xy')
df_axis0 = pd.concat([df1, df2])
df_axis1 = pd.concat([df1, df2], axis=1)
display('df1', 'df2' , 'df_axis0' , 'df_axis1')
```

Out[118…

df1

	A	B
x	Ax	Bx
y	Ay	By

df2

	A	B
x	Ax	Bx
y	Ay	By

df_axis0

	A	B
x	Ax	Bx
y	Ay	By
x	Ax	Bx
y	Ay	By

df_axis1

	A	B	A	B
x	Ax	Bx	Ax	Bx
y	Ay	By	Ay	By

- `verify_integrity=True` indizeak bikoiztea eragozten du (`ValueError`)

In [119…

```
df1 = make_df('AB', 'xy')
df2 = make_df('AB', 'xy')
try:
    pd.concat([df1, df2], verify_integrity=True)
except ValueError as e:
    print("axis=0 ValueError:", e)
try:
    pd.concat([df1, df2], axis=1, verify_integrity=True)
except ValueError as e:
    print("axis=1 ValueError:", e)
display('df1', 'df2')
```

```
axis=0 ValueError: Indexes have overlapping values: Index(['x', 'y'], dtype='object')
axis=1 ValueError: Indexes have overlapping values: Index(['A', 'B'], dtype='object')
```

Out[119…

df1

	A	B
x	Ax	Bx
y	Ay	By

df2

	A	B
x	Ax	Bx
y	Ay	By

- `ignore_index=True` gainjarritako indizeak baztertzen ditu (`RangeIndex` -ekin ordezkatzen dira)

In [120…

```
df1 = make_df('AB', 'xy')
df2 = make_df('AB', 'xy')
df_axis0 = pd.concat([df1, df2], ignore_index=True)
df_axis1 = pd.concat([df1, df2], axis=1, ignore_index=True)
pprint('df_axis0.index', 'df_axis0.columns',
```

```
      'df_axis1.index', 'df_axis1.columns', align=True)
display('df1', 'df2' , 'df_axis0' , 'df_axis1')
```

```
  df_axis0.index = RangeIndex(start=0, stop=4, step=1)
df_axis0.columns = Index(['A', 'B'], dtype='object')
  df_axis1.index = Index(['x', 'y'], dtype='object')
df_axis1.columns = RangeIndex(start=0, stop=4, step=1)
```

Out[120…

df1

	A	B
x	Ax	Bx
y	Ay	By

df2

	A	B
x	Ax	Bx
y	Ay	By

df_axis0

	A	B
0	Ax	Bx
1	Ay	By
2	Ax	Bx
3	Ay	By

df_axis1

	0	1	2	3
x	Ax	Bx	Ax	Bx
y	Ay	By	Ay	By

- `join="outer"` (defektuz): datu nuluak sor ditzake
- `join="inner` : datu nuluak saihesten ditu
  - `axis=0` → zutabeak kentzen ditu
  - `axis=1` → errenkadak kentzen ditu

In [121…

```
df1 = make_df('ABC', [1,2])
df2 = make_df('BCD', [3,4])
display('df1', 'df2')
```

Out[121…

df1

	A	B	C
1	A1	B1	C1
2	A2	B2	C2

df2

	B	C	D
3	B3	C3	D3
4	B4	C4	D4

In [122…

```
df_outer = pd.concat([df1, df2])
df_inner = pd.concat([df1, df2], join='inner')
display('df_outer', 'df_inner')
```

Out[122…

df_outer

	A	B	C	D
1	A1	B1	C1	NaN
2	A2	B2	C2	NaN
3	NaN	B3	C3	D3
4	NaN	B4	C4	D4

df_inner

	B	C
1	B1	C1
2	B2	C2
3	B3	C3
4	B4	C4

In [123…

```
df1 = make_df('AB', [1,2,3])
df2 = make_df('CD', [2,3,4])
display('df1', 'df2')
```

Out[123…

df1

	A	B
1	A1	B1
2	A2	B2
3	A3	B3

df2

	C	D
2	C2	D2
3	C3	D3
4	C4	D4

In [124…

```
df_outer = pd.concat([df1, df2], axis=1)
df_inner = pd.concat([df1, df2], axis=1, join='inner')
display('df_outer', 'df_inner')
```

Out[124…

df_outer

	A	B	C	D
1	A1	B1	NaN	NaN
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	NaN	NaN	C4	D4

df_inner

	A	B	C	D
2	A2	B2	C2	D2
3	A3	B3	C3	D3

Dataset-ak bateratzen

`pd.merge()`

- Bi Series/DataFrame bateratzen ditu
- Zutabe *komunak* bateratze-gako gisa erabiltzen dira

```
pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None,
         left_index=False, right_index=False, sort=False,
         suffixes=('_x', '_y'), copy=None, indicator=False, validate=None)
```

In [125…

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa'],
                    'group': ['Accounting', 'Engineering','Engineering']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake'],
                    'hire_date': [2004, 2008, 2012]})
display('df1', 'df2')
```

Out[125…

df1

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering

df2

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012

In [126…

```
pd.merge(df1, df2)
```

Out[126…

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004

In [127…

```
df1 = pd.merge(df1, df2)
df2 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                    'supervisor': ['Carly', 'Guido', 'Steve']})
display('df1', 'df2')
```

Out[127…

df1

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004

df2

	group	supervisor
0	Accounting	Carly
1	Engineering	Guido
2	HR	Steve

In [128…

```
pd.merge(df1, df2)
```

Out[128…

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido

In [129…

```
df2 = pd.DataFrame({'group': ['Accounting', 'Accounting', 'Engineering', 'Engineering']
                    'skills': ['math', 'spreadsheets', 'software', 'math']})
display('df1', 'df2')
```

Out[129…

df1

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004

df2

	group	skills
0	Accounting	math
1	Accounting	spreadsheets
2	Engineering	software
3	Engineering	math

In [130…

```
pd.merge(df1, df2)
```

Out[130…

	employee	group	hire_date	skills
0	Bob	Accounting	2008	math
1	Bob	Accounting	2008	spreadsheets
2	Jake	Engineering	2012	software
3	Jake	Engineering	2012	math
4	Lisa	Engineering	2004	software
5	Lisa	Engineering	2004	math

- `on=column_name` → zutabea gako gisa erabili
- `left_on=name1`, `right_on=name2` → zutabeak gako gisa erabili

In [131…

```
df2 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'], 'salary': [70000, 80000, 12
display('df1', 'df2')
```

Out[131…

df1

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004

df2

	name	salary
0	Bob	70000
1	Jake	80000
2	Lisa	120000
3	Sue	90000

In [132…

```
pd.merge(df1, df2, left_on="employee", right_on="name")
```

Out[132…

	employee	group	hire_date	name	salary
0	Bob	Accounting	2008	Bob	70000
1	Jake	Engineering	2012	Jake	80000
2	Lisa	Engineering	2004	Lisa	120000

`left_on=name1`, `right_on=name2` erabiltzeak zutabe erredundante bat sortzen du

In [133…

```
pd.merge(df1, df2, left_on="employee", right_on="name")
```

Out[133…

	employee	group	hire_date	name	salary
0	Bob	Accounting	2008	Bob	70000
1	Jake	Engineering	2012	Jake	80000
2	Lisa	Engineering	2004	Lisa	120000

In [134…

```
pd.merge(df1, df2, left_on="employee", right_on="name").drop('name', axis=1)
```

Out[134…

	employee	group	hire_date	salary
0	Bob	Accounting	2008	70000
1	Jake	Engineering	2012	80000
2	Lisa	Engineering	2004	120000

- `left_index=True` , `right_index=True` → indizeak gako gisa erabili

In [135…

```
df2 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa'],
                    'salary': [70000, 80000, 120000]})
df2.set_index('name', inplace=True)
display('df1', 'df2')
```

Out[135…

df1

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004

df2

	salary
name	
Bob	70000
Jake	80000
Lisa	120000

In [136…

```
pd.merge(df1, df2, left_on='employee', right_index=True)
```

Out[136…

	employee	group	hire_date	salary
0	Bob	Accounting	2008	70000
1	Jake	Engineering	2012	80000
2	Lisa	Engineering	2004	120000

- `df1.join(df2)` → indizeetan oinarritutako bateratzea
  - `pd.merge(df1, df2, left_index=True, right_index=True)`

In [137…

```
df1.set_index('employee', inplace=True)
display('df1', 'df2')
```

Out[137…

df1

	group	hire_date
employee		
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004

df2

	salary
name	
Bob	70000
Jake	80000
Lisa	120000

In [138…

```
df1.join(df2)
```

Out[138…

employee	group	hire_date	salary
Bob	Accounting	2008	70000
Jake	Engineering	2012	80000
Lisa	Engineering	2004	120000

- `how='inner'` (lehenetsia) → ebakidura

In [139…

```
df1 = pd.DataFrame({'name': ['Peter', 'Paul', 'Mary', 'Adam'],
                    'food': ['fish', 'beans', 'bread' , 'fish']})
df2 = pd.DataFrame({'name': ['Mary', 'Joseph', 'Peter' , 'Alice'],
                    'drink': ['wine', 'beer' , 'water' , 'beer']})
display('df1', 'df2', 'pd.merge(df1, df2)')
```

Out[139…

df1

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread
3	Adam	fish

df2

	name	drink
0	Mary	wine
1	Joseph	beer
2	Peter	water
3	Alice	beer

pd.merge(df1, df2)

	name	food	drink
0	Peter	fish	water
1	Mary	bread	wine

- `how='outer'` → bilketa (balio nuluak)

In [140…

```
display('df1', 'df2', 'pd.merge(df1, df2, how="outer")')
```

Out[140…

df1

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread
3	Adam	fish

df2

	name	drink
0	Mary	wine
1	Joseph	beer
2	Peter	water
3	Alice	beer

pd.merge(df1, df2, how="outer")

	name	food	drink
0	Adam	fish	NaN
1	Alice	NaN	beer
2	Joseph	NaN	beer
3	Mary	bread	wine
4	Paul	beans	NaN
5	Peter	fish	water

- `how='left'` → mantendu ezkerreko gakoa (balio nuluak)

In [141…

```
display('df1', 'df2', 'pd.merge(df1, df2, how="left")')
```

Out[141…

df1

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread
3	Adam	fish

df2

	name	drink
0	Mary	wine
1	Joseph	beer
2	Peter	water
3	Alice	beer

pd.merge(df1, df2, how="left")

	name	food	drink
0	Peter	fish	water
1	Paul	beans	NaN
2	Mary	bread	wine
3	Adam	fish	NaN

- `how='right'` → mantendu eskuineko gakoa (balio nuluak)

In [142…

```
display('df1', 'df2', 'pd.merge(df1, df2, how="right")')
```

Out[142…

df1

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread
3	Adam	fish

df2

	name	drink
0	Mary	wine
1	Joseph	beer
2	Peter	water
3	Alice	beer

pd.merge(df1, df2, how="right")

	name	food	drink
0	Mary	bread	wine
1	Joseph	NaN	beer
2	Peter	fish	water
3	Alice	NaN	beer

Agregazioa eta taldekatzea

Planeten datuak

Astronomoek aurkitutako exoplanetei buruzko informazioa (eguzkia ez den beste izar baten inguruan orbitatzen duten planetak).

In [143…

```
import seaborn as sns
planets = sns.load_dataset('planets')
planets.shape
```

Out[143…

```
(1035, 6)
```

In [144…

```
planets.head(10)  #planets[:10]
```

Out[144...

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009
5	Radial Velocity	1	185.840	4.80	76.39	2008
6	Radial Velocity	1	1773.400	4.64	18.15	2002
7	Radial Velocity	1	798.500	NaN	21.41	1996
8	Radial Velocity	1	993.300	10.30	73.10	2008
9	Radial Velocity	2	452.800	1.99	74.79	2010

In [145...

```
planets.dropna(inplace=True)
print(planets.shape)
planets.head(10)
```

```
(498, 6)
```

Out[145...

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009
5	Radial Velocity	1	185.840	4.80	76.39	2008
6	Radial Velocity	1	1773.400	4.64	18.15	2002
8	Radial Velocity	1	993.300	10.30	73.10	2008
9	Radial Velocity	2	452.800	1.99	74.79	2010
10	Radial Velocity	2	883.000	0.86	74.79	2010

Agregazio-funtzioak

Pandas-en berezko (*built-in*) agregazio batzuk:

Agregazioa	Deskribapena
`count()`	Elementu kopurua
`first()`, `last()`	Lehen eta azken elementua
`mean()`, `median()`	Batezbestekoa eta mediana
`min()`, `max()`	Minimoa eta maximoa
`std()`, `var()`	Desbideratze estandarra eta bariantza
`mad()`	Batez besteko desbideratze absolutua

Agregazioa	Deskribapena
`sum()`, `prod()`	Elementu guztien batura eta biderkadura

In [146…

```
planets = sns.load_dataset('planets')
df = planets.dropna().drop(['method','number'], axis=1)
print(df.shape)
pprint('df.min()', 'df.max()', 'df.mean()', 'df.std()', align=True)
```

```
(498, 4)
 df.min() = orbital_period       1.3283
            mass                 0.0036
            distance             1.3500
            year              1989.0000
            dtype: float64
 df.max() = orbital_period    17337.5
            mass                 25.0
            distance            354.0
            year               2014.0
            dtype: float64
df.mean() = orbital_period     835.778671
            mass                 2.509320
            distance            52.068213
            year              2007.377510
            dtype: float64
 df.std() = orbital_period    1469.128259
            mass                 3.636274
            distance            46.596041
            year                 4.167284
            dtype: float64
```

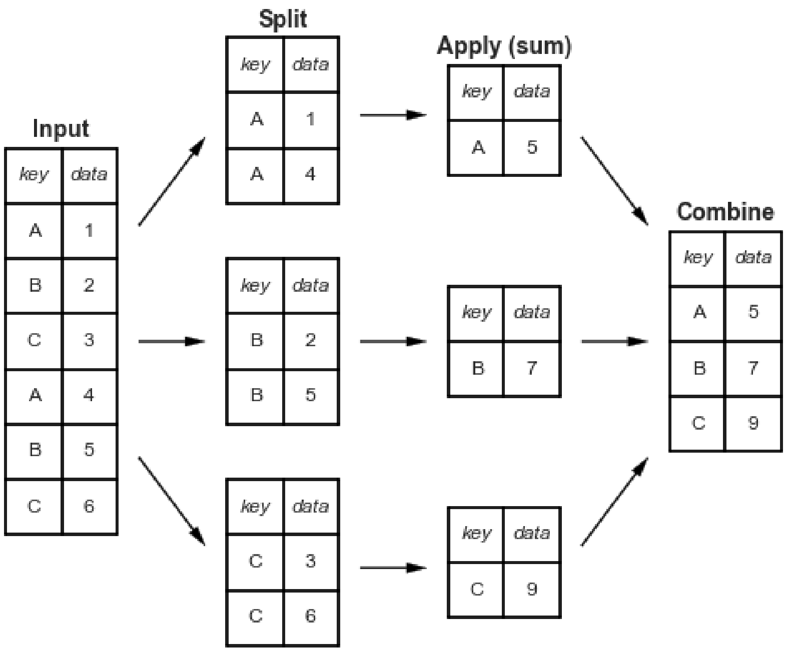
In [147…

```
df = planets.drop(['method','number'], axis=1)
print(df.shape)
pprint('df.min()', 'df.max()', 'df.mean()', 'df.std()')
```

```
(1035, 4)
df.min() = orbital_period       0.090706
           mass                 0.003600
           distance             1.350000
           year              1989.000000
           dtype: float64
df.max() = orbital_period    730000.0
           mass                  25.0
           distance            8500.0
           year                2014.0
           dtype: float64
df.mean() = orbital_period    2002.917596
            mass                 2.638161
            distance           264.069282
            year              2009.070531
            dtype: float64
df.std() = orbital_period    26014.728304
           mass                  3.818617
           distance            733.116493
           year                  3.972567
           dtype: float64
```

GroupBy

- `df.groupby()` → DataFrame *bilduma* bat
    - DataFrame baten bista berezi moduko bat
- DataFrame bat bezala indexa daiteke → GroupBy objektu berri bat
- *zatitu-aplikatu-konbinatu* motako eragiketa errazak eskaintzen ditu



In [148…

```
planets.groupby('method')
```

Out[148…

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7f1fe561f140>
```

In [149…

```
planets.groupby('method')['orbital_period']
```

Out[149…

```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x7f1b996c9b80>
```

In [150…

```
planets.groupby('method')['orbital_period'].median()
```

Out[150…

```
method
Astrometry                         631.180000
Eclipse Timing Variations         4343.500000
Imaging                          27500.000000
Microlensing                      3300.000000
Orbital Brightness Modulation        0.342887
Pulsar Timing                       66.541900
Pulsation Timing Variations       1170.000000
Radial Velocity                    360.200000
Transit                              5.714932
Transit Timing Variations           57.011000
Name: orbital_period, dtype: float64
```

GroupBy objektuak esplizituki inplementatzen ez dituen metodoak taldeetara pasatu eta haietan ebaluatzen dira.

In [151…

```
planets['orbital_period'].describe()
```

Out[151…
```
count        992.000000
mean        2002.917596
std        26014.728304
min            0.090706
25%            5.442540
50%           39.979500
75%          526.005000
max       730000.000000
Name: orbital_period, dtype: float64
```

In [152…
```
planets.groupby('method')['orbital_period'].describe()
```

Out[152…

method	count	mean	std	min	25%	50%	75%	
Astrometry	2.0	631.180000	544.217663	246.360000	438.770000	631.180000	823.590000	1
Eclipse Timing Variations	9.0	4751.644444	2499.130945	1916.250000	2900.000000	4343.500000	5767.000000	10
Imaging	12.0	118247.737500	213978.177277	4639.150000	8343.900000	27500.000000	94250.000000	730
Microlensing	7.0	3153.571429	1113.166333	1825.000000	2375.000000	3300.000000	3550.000000	5
Orbital Brightness Modulation	3.0	0.709307	0.725493	0.240104	0.291496	0.342887	0.943908	
Pulsar Timing	5.0	7343.021201	16313.265573	0.090706	25.262000	66.541900	98.211400	36
Pulsation Timing Variations	1.0	1170.000000	NaN	1170.000000	1170.000000	1170.000000	1170.000000	1
Radial Velocity	553.0	823.354680	1454.926210	0.736540	38.021000	360.200000	982.000000	17
Transit	397.0	21.102073	46.185893	0.355000	3.160630	5.714932	16.145700	
Transit Timing Variations	3.0	79.783500	71.599884	22.339500	39.675250	57.011000	108.505500	

GroupBy objektua iteragarria da

- `(groupby_value , Series/DataFrame)` bikote sekuentzia bat

In [153…
```
for method,df in planets.groupby('method'):
    print(f'{method:30s} {df.shape=}')
```

```
Astrometry                     df.shape=(2, 6)
Eclipse Timing Variations      df.shape=(9, 6)
Imaging                        df.shape=(38, 6)
Microlensing                   df.shape=(23, 6)
Orbital Brightness Modulation  df.shape=(3, 6)
Pulsar Timing                  df.shape=(5, 6)
Pulsation Timing Variations    df.shape=(1, 6)
Radial Velocity                df.shape=(553, 6)
Transit                        df.shape=(397, 6)
Transit Timing Variations      df.shape=(4, 6)
```

Agregatu, iragazi, transformatu eta aplikatu

GroupBy objektuek `aggregate()`, `filter()`, `transform()`, eta `apply()` metodoak dituzte; metodo horiek hainbat eragiketa erabilgarri modu eraginkorrean inplementatzen dituzte taldekatutako datuak konbinatu aurretik.

In [154…
```
planets.groupby('method')[['orbital_period','distance']].aggregate(['count','min','max'
```

Out[154…

	orbital_period			distance		
	count	min	max	count	min	max
method						
Astrometry	2	246.360000	1016.000000	2	14.98	20.77
Eclipse Timing Variations	9	1916.250000	10220.000000	4	130.72	500.00
Imaging	12	4639.150000	730000.000000	32	7.69	165.00
Microlensing	7	1825.000000	5100.000000	10	1760.00	7720.00
Orbital Brightness Modulation	3	0.240104	1.544929	2	1180.00	1180.00
Pulsar Timing	5	0.090706	36525.000000	1	1200.00	1200.00
Pulsation Timing Variations	1	1170.000000	1170.000000	0	NaN	NaN
Radial Velocity	553	0.736540	17337.500000	530	1.35	354.00
Transit	397	0.355000	331.600590	224	38.00	8500.00
Transit Timing Variations	3	22.339500	160.000000	3	339.00	2119.00

Denbora-serieekin lan egiten

- Pandas datekin, instanteekin eta denboraren arabera indexatutako datuekin bateragarria da
- Denboran oinarritutako indize bat erabil daiteke denbora-serietarako
- Indexazio estandarra eta datei dagokien indexazio berezia

Python-eko datak eta instanteak: `datetime` and `dateutil`

In [155…
```
from datetime import datetime
date = datetime(year=2025, month=4, day=14)
date
```

Out[155…
```
datetime.datetime(2025, 4, 14, 0, 0)
```

`dateutil` moduluak hainbat formatutako karaktere-kateak ulertzen ditu:

In [156…
```
from dateutil import parser
date = parser.parse("14th of April, 2025")
date
```

Out[156…
```
datetime.datetime(2025, 4, 14, 0, 0)
```

NumPy-ren `datetime64`

In [157…
```
import numpy as np
date = np.array('2025-04-14', dtype=np.datetime64)
date
```

Out[157…
```
array('2025-04-14', dtype='datetime64[D]')
```

Eragiketa bektorizatuak datekin erabil daitezke:

In [158…
```
date + np.arange(20)
```

Out[158…
```
array(['2025-04-14', '2025-04-15', '2025-04-16', '2025-04-17',
       '2025-04-18', '2025-04-19', '2025-04-20', '2025-04-21',
       '2025-04-22', '2025-04-23', '2025-04-24', '2025-04-25',
       '2025-04-26', '2025-04-27', '2025-04-28', '2025-04-29',
       '2025-04-30', '2025-05-01', '2025-05-02', '2025-05-03'],
      dtype='datetime64[D]')
```

Datak eta instanteak Pandas-en

In [159…
```
date = pd.to_datetime("4th of July, 2015")
date
```

Out[159…
```
Timestamp('2015-07-04 00:00:00')
```

Eragiketa bektorizatuak datekin erabil daitezke:

In [160…
```
date + pd.to_timedelta(np.arange(20), 'D')
```

Out[160…
```
DatetimeIndex(['2015-07-04', '2015-07-05', '2015-07-06', '2015-07-07',
               '2015-07-08', '2015-07-09', '2015-07-10', '2015-07-11',
               '2015-07-12', '2015-07-13', '2015-07-14', '2015-07-15',
               '2015-07-16', '2015-07-17', '2015-07-18', '2015-07-19',
               '2015-07-20', '2015-07-21', '2015-07-22', '2015-07-23'],
              dtype='datetime64[ns]', freq=None)
```

Datak eta instanteak Pandas-en

`DatetimeIndex` -ek denbora-indize bat eskaintzen du

In [161…
```
index = pd.DatetimeIndex(['2014-07-04', '2014-08-04',
                          '2015-07-04', '2015-08-04'])
data = pd.Series([0, 1, 2, 3], index=index)
data
```

Out[161…
```
2014-07-04    0
2014-08-04    1
2015-07-04    2
2015-08-04    3
dtype: int64
```

Indexazio-mota estandarrak:

In [162…
```
pprint("data['2014-08-04']", "data[:'2015-07-04']", "data[['2014-07-04','2015-07-04']]'
```
```
              data['2014-08-04'] = 1

             data[:'2015-07-04'] = 2014-07-04    0
                                   2014-08-04    1
                                   2015-07-04    2
                                   dtype: int64

data[['2014-07-04','2015-07-04']] = 2014-07-04    0
                                   2015-07-04    2
                                   dtype: int64
```

Datetan oinarritutako indexazioak:

In [163…
```
pprint("data['2014']", "data[:'2015-07']", align=True, sep="\n\n")
```
```
   data['2014'] = 2014-07-04    0
                  2014-08-04    1
                  dtype: int64

data[:'2015-07'] = 2014-07-04    0
                  2014-08-04    1
                  2015-07-04    2
                  dtype: int64
```

Denbora-serieen datu-egiturak

- *denbora-zigiluak*: `Timestamp` (np.datetime64) → `DatetimeIndex`
- *denbora-tarteak*: `Period` (np.datetime64) → `PeriodIndex`
- *denbora-diferentziak edo iraupenak*: `Timedelta` (np.timedelta64) → `TimedeltaIndex`

- `pd.to_datetime()` (analizatzailea, *parser*) → `Timestamp` or `DatetimeIndex`

In [164…
```
dates = pd.to_datetime([datetime(2015, 7, 3), '4th of July, 2015',
                        '2015-Jul-6', '07-07-2015', '20150708'])
pprint('dates','dates[0]', align=True)
```
```
   dates = DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07',
                          '2015-07-08'],
                         dtype='datetime64[ns]', freq=None)
dates[0] = Timestamp('2015-07-03 00:00:00')
```

- `DatetimeIndex.to_period()` → `PeriodIndex`

In [165…
```
dates.to_period('D')
```
Out[165…
```
PeriodIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07',
             '2015-07-08'],
            dtype='period[D]')
```

In [166…
```
days = dates.to_period('D')
months = dates.to_period('M')
```

```python
years = dates.to_period('Y')
pprint('dates[:2]', 'days[:2]', 'months[:2]', 'years[:2]', align=True)
```

```
 dates[:2] = DatetimeIndex(['2015-07-03', '2015-07-04'], dtype='datetime64[ns]', freq=N
one)
  days[:2] = PeriodIndex(['2015-07-03', '2015-07-04'], dtype='period[D]')
months[:2] = PeriodIndex(['2015-07', '2015-07'], dtype='period[M]')
 years[:2] = PeriodIndex(['2015', '2015'], dtype='period[Y-DEC]')
```

- `Timestamp - Timestamp` → `Timedelta`:

In [167…

```python
dates[1]-dates[0]
```

Out[167…

```
Timedelta('1 days 00:00:00')
```

- `DatetimeIndex - Timestamp` → `TimedeltaIndex` :

In [168…

```python
dates-dates[0]
```

Out[168…

```
TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'], dtype='timedelta64
[ns]', freq=None)
```

- `pd.date_range()` → `DatetimeIndex`
- `pd.period_range()` → `PeriodIndex`
- `pd.timedelta_range()` → `TimedeltaIndex`

In [169…

```python
pd.date_range('2015-07-03', '2015-07-10')
```

Out[169…

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',
               '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],
              dtype='datetime64[ns]', freq='D')
```

In [170…

```python
pd.date_range('2015-07-03', periods=8, freq='h')
```

Out[170…

```
DatetimeIndex(['2015-07-03 00:00:00', '2015-07-03 01:00:00',
               '2015-07-03 02:00:00', '2015-07-03 03:00:00',
               '2015-07-03 04:00:00', '2015-07-03 05:00:00',
               '2015-07-03 06:00:00', '2015-07-03 07:00:00'],
              dtype='datetime64[ns]', freq='h')
```

In [171…

```python
pd.period_range('2015-07', periods=8, freq='M')
```

Out[171…

```
PeriodIndex(['2015-07', '2015-08', '2015-09', '2015-10', '2015-11', '2015-12',
             '2016-01', '2016-02'],
            dtype='period[M]')
```

In [172…

```python
pd.timedelta_range(0, periods=10, freq='h')
```

Out[172…

```
TimedeltaIndex(['0 days 00:00:00', '0 days 01:00:00', '0 days 02:00:00',
                '0 days 03:00:00', '0 days 04:00:00', '0 days 05:00:00',
                '0 days 06:00:00', '0 days 07:00:00', '0 days 08:00:00',
                '0 days 09:00:00'],
               dtype='timedelta64[ns]', freq='h')
```

Ber-lagintzea (*Resampling*), desplazatzea (*Shifting*) eta leihokatzea (*Windowing*)

In [ ]:
```
#!pip install pandas-datareader
from pandas_datareader import data
#goog = data.DataReader('GOOG', start=2020, end='2024',data_source='stooq')
goog = data.DataReader('GOOG', start=2020, end='2024', data_source='stooq').sort_index(
goog.head()
```

onfigurazio bisual batzuk...

In [ ]:
```
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn
seaborn.set()
mpl.rcParams['figure.figsize'] = (5.33,4)
mpl.rcParams['axes.labelsize'] = 10  # Adibidea: 14 puntu
mpl.rcParams['xtick.labelsize'] = 8  # Adibidea: 12 puntu x ardatzeko marketarako
mpl.rcParams['ytick.labelsize'] = 8  # Adibidea: 12 puntu y ardatzeko marketarako
```

In [ ]:
```
goog = goog['Close']
```

In [ ]:
```
goog.plot(linewidth=0.5);
```

- `resample(freq)` → agregazio/murrizketa eragiketa baterako prest dagoen objektu taldekatua
- `resample(freq).aggregate_func()` → agregatutako balioak dituen denbora-serie berria
- `asfreq(freq)` → hautatutako/aldizkako balioak dituen denbora-serie berria
  - ≈ `resample(freq).last()`

In [ ]:
```
# 'BYE': Negozio-urtearen amaiera (urteko azken laneguna)
goog.plot(alpha=0.5, style='-', linewidth=0.5)
goog.resample('BYE').mean().plot(style=':')
goog.asfreq('BYE').plot(style='--');
plt.legend(['original', 'resample', 'asfreq'], loc='upper left');
```

Berriz lagintzeak (`resample+aggregation` edo `asfreq`) balio nuluak sor ditzake

- `method="ffill"|"bfill"` → balioak nola egozten diren

In [ ]:
```
d = goog.head()
pprint("d" , "d.asfreq('D')", "d.asfreq('D',method='ffill')", align=True)
```

In [ ]:
```
fig, ax = plt.subplots(2, sharex=True)
data = goog.head()

data.asfreq('D').plot(ax=ax[0], marker='o')

data.asfreq('D', method='bfill').plot(ax=ax[1], style='-o')
data.asfreq('D', method='ffill').plot(ax=ax[1], style='--o')
ax[1].legend(["back-fill", "forward-fill"]);
```

- `shift()` → datuak desplazatu

- `shift(int)` → datuak `int` posizio desplazatu
- `shift(int, freq)` → datuak `freq` -eko `int` kopurua desplazatu

In [ ]:
```
pprint("goog.head()", "goog.shift(3).head()", "goog.shift(3,'D').head()", align=True)
```

Kalkulatu denboran zeharreko diferentziak (maiztasunak):

In [ ]:
```
pprint("goog.head()", "goog.shift(-365,'D').head()", align=True)
```

In [ ]:
```
data = goog.asfreq('D', method='ffill')
ROI = 100 * (data.shift(-365,'D') / data - 1)
ROI.plot()
plt.ylabel('% Return on Investment');
```

- `rolling()` → leiho mugikorra
  - agregazio/murrizketa funtzioak leiho mugikor bati aplika dakizkioke

In [ ]:
```
r365 = goog.rolling(365).mean()
r200 = goog.rolling(200).mean()
data = pd.DataFrame({'GOOG': goog, '1year mean': r365, '200d mean': r200})
ax = data.plot(style=['-', '--', ':'])
ax.lines[0].set_alpha(0.3)
```

`rolling(365)` ez da urtebeteko leiho mugikorra...

In [ ]:
```
r365 = goog.asfreq('D', method='ffill').rolling(365).mean()
r200 = goog.asfreq('D', method='ffill').rolling(200).mean()
data = pd.DataFrame({'GOOG': goog, '1year mean': r365, '200d mean': r200})
ax = data.plot(style=['-', '--', ':'])
ax.lines[0].set_alpha(0.3)
```